

INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Tabla genérica

Dr. Luciano H. Tamargo
http://cs.uns.edu.ar/~lt
Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

GENERICIDAD

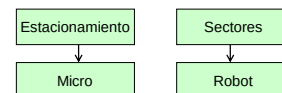
- Una **clase genérica** encapsula a una estructura cuyo comportamiento es independiente del tipo de las componentes.
- Las clases genéricas favorecen la **reusabilidad** y la **extensibilidad**.
- La genericidad puede modelarse en Java de dos maneras diferentes: usando **polimorfismo paramétrico** o usando **herencia**.
- En esta materia definiremos clases genéricas usando herencia.

TEMARIO

- Caso de estudio: Sin genericidad.
- Caso de estudio: Con genericidad.

CASO DE ESTUDIO: SIN GENERICIDAD

- Representaremos el conjunto de vehículos de una cochera con una clase **Estacionamiento** que encapsula a un arreglo de componentes de clase **Micro** y brinda operaciones para estacionar, retirar, buscar, etc.
- Representamos a un conjunto de robots de una fábrica con una clase **Sectores** que encapsula un arreglo de componentes de clase **Robot** y brinda operaciones para asignar, desasignar, buscar, etc.



Estacionamiento	Asigna unMicro en la Unidad <i>u</i> . Requiere controlada la unidad y micro no nulo.
t: arreglo de Micro	
<<Constructores>>	
Estacionamiento(max: entero)	
<<Comandos>>	
estacionar(unMicro: Micro , u: entero)	Busca la primera unidad libre y asigna unMicro a la unidad. Requiere que haya una unidad libre y unMicro no sea nulo.
estacionar(unMicro: Micro)	
retirar(u: entero)	Elimina unMicro de la unidad <i>u</i> . Requiere controlada la unidad.
<<Consultas>>	
cantUnidades(): entero	
cantUnidadesOcupadas(): entero	
todosReservados(): boolean	
estaMicro(unMicro: Micro): boolean	Retorna true si en el estacionamiento hay un micro estacionado con la misma identidad que unMicro
existeUnidad(u: entero): boolean	
microUnidad(u: entero): Micro	

CASO DE ESTUDIO: SIN GENERICIDAD

```

class Estacionamiento {
    private Micro[] t;

    //Constructor
    public Estacionamiento(int max) {
        /*Crea una Tabla con capacidad para max elementos. */
        t = new Micro[max];
    }
    ...
}
  
```

CASO DE ESTUDIO: SIN GENERICIDAD

```
//Comandos
public void estacionar(Micro unMicro, int u) {
    /*Asigna unMicro a la unidad u, requiere controlada la unidad y
    unMicro no nulo*/
    t[u] = unMicro;
}

public void estacionar(Micro unMicro) {
    /*Busca la primera posición libre e inserta unMicro. La clase
    Cliente es responsable de controlar que la tabla no esté llena y
    unMicro no sea nulo*/
    int i = 0;
    while (t[i] != null)
        i++;
    t[i] = unMicro;
}
```

CASO DE ESTUDIO: SIN GENERICIDAD

```
public void retirar(int u){
    t[u] = null;
}

//Consultas
public int cantUnidades(){
    return t.length;
}

public int cantUnidadesOcupadas(){
    int cant = 0;
    for (int i = 0; i < cantUnidades(); i++)
        if (t[i] != null)
            cant++;
    return cant;
}
```

CASO DE ESTUDIO: SIN GENERICIDAD

```
public boolean todosReservados() {
    /*Retorna true si todos los micros tienen todos los asientos
    reservados*/
    boolean todos = true;
    for (int i = 0; i < cantUnidades() && todos; i++)
        if (t[i] != null)
            todos = !t[i].hayAsientosLibres();
    return todos;
}
```

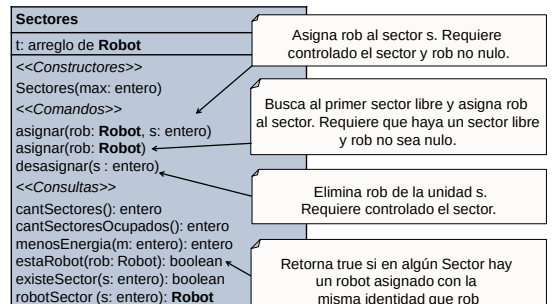
CASO DE ESTUDIO: SIN GENERICIDAD

```
public boolean estaMicro(Micro unMicro){
    /*Decide si alguna unidad está ligada al micro unMicro, que asume
    no nulo*/
    boolean esta = false;
    for (int i = 0; i < cantUnidades() && !esta; i++)
        esta = t[i] == unMicro;
    return esta;
}

public boolean existeUnidad(int u){
    /*Decide si existe una unidad de estacionamiento u*/
    return (u >= 0 & u < cantUnidades());
}

public Micro microUnidad(int u){
    /*Retorna el micro de la unidad u. Requiere u válida*/
    return t[u];
}
```

```
class Parking{
    private Estacionamiento s;
    ...
    public Parking(){
        s = new Estacionamiento(50);
    }
    ...
    public void administrar(){
        ...
        int pos;
        ...
        pos = 0;
        Micro r = new Micro(...);
        s.estacionar(r, pos);
    }
    ...
}
```



CASO DE ESTUDIO: SIN GENERICIDAD

```
class Sectores {
    private Robot[] t;

    //Constructor
    public Sectores(int max) {
        /*Crea una Tabla con capacidad para max elementos. */
        t = new Robot[max];
    }
    ...
}
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13

CASO DE ESTUDIO: SIN GENERICIDAD

```
//Comandos
public void asignar(Robot rob, int s){
    /*Asigna el robot rob al sector s, requiere controlado el
    sector y rob no nulo*/
    t[s] = rob;
}

public void asignar(Robot rob) {
    /*Busca el primer sector libre y asigna el robot rob. La
    clase Cliente es responsable de controlar que la tabla no
    este llena y rob no sea nulo*/
    int i = 0;
    while (t[i] != null)
        i++;
    t[i] = rob;
}
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14

CASO DE ESTUDIO: SIN GENERICIDAD

```
public void desasignar(int s){
    t[s] = null;
}

//Consultas
public int cantSectores(){
    return t.length;
}

public int cantSectoresOcupados(){
    int cant = 0;
    for (int i = 0; i < cantSectores(); i++){
        if (t[i] != null)
            cant++;
    }
    return cant;
}
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

CASO DE ESTUDIO: SIN GENERICIDAD

```
public int menosEnergia(int m) {
    /*Retorna la cantidad de robots que tienen energia menor a
    m*/
    int cant = 0;
    for (int i = 0; i < cantSectores(); i++){
        if (t[i] != null && t[i].obtenerEnergia() < m)
            cant++;
    }
    return cant;
}
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

CASO DE ESTUDIO: SIN GENERICIDAD

```
public boolean estaRobot(Robot rob){
    /*Decide si algún sector está ligado al robot rob, que asume
    no nulo*/
    boolean esta = false;
    for (int i = 0; i < cantSectores() && !esta; i++)
        esta = t[i] == rob;
    return esta;
}
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

- Busca un robot con la misma **identidad** que el parámetro formal **rob**.

CASO DE ESTUDIO: SIN GENERICIDAD

```
public boolean existeSector(int s){
    /*Decide si existe un sector s*/
    return (s >= 0 & s < cantSectores());
}

public Robot robotSector(int s){
    /*Retorna el robot del sector s. Requiere s válido*/
    return t[s];
}
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18

```
class FabricaJuguetes{
    private Sectores s;
    ...
    public FabricaJuguetes() {
        s = new Sectores (70);
    }
    public void producir () {
        ...
        int pos;
        ...
        pos = 0;
        Robot r = new Robot ("Tommy");
        s.asignar(r, pos);
        ...
    }
    ...
}
```

TEMARIO

- Caso de estudio: Sin genericidad.
- Caso de estudio: Con genericidad.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
1111
000
1
```

Tabla	
t: arreglo de Object	
<<Constructores>>	
Tabla(max: entero)	
<<Comandos>>	
insertar(elem: Object, p: entero)	Inserta elem en la posición p. Requiere controlada la posición y elem no nulo.
eliminar(elem: Object)	Busca la primera posición libre y asigna el elem a la posición. Requiere que haya una posición libre y elem no sea nulo.
eliminar(p: entero)	Elimina elem de la posición p. Requiere controlada la posición.
<<Consultas>>	
cantPosiciones(): entero	
cantPosicionesOcupadas(): entero	
estaElemento(elem: Object): boolean	
existePosicion(p: entero): boolean	
elementoPosicion(p: entero): Object	Retorna true si algún elemento de la tabla tiene la misma identidad que elem

GENERICIDAD

```
class Tabla{
    protected Object[] t;

    //Constructor
    public Tabla(int max){
        /*Crea una Tabla con capacidad para max elementos. */
        t = new Object[max];
    }
    ...
}
```

```
0
1
0
0
0
0
1
0
0
```

GENERICIDAD

```
//Comandos
public void insertar(Object elem){
    /*Busca la primera posición libre e inserta. La clase
    Cliente es responsable de controlar que la tabla no esté
    llena y elem no sea nulo*/
    int i = 0;
    while (t[i] != null)
        i++;
    t[i] = elem;
}
```

GENERICIDAD

```
public void insertar(Object elem, int p){
    /*Inserta un elemento en la posición p, requiere controlada
    la posición y elem no nulo*/
    t[p] = elem;
}

public void eliminar(int p){
    t[p] = null;
}
```

```
0
1
0
0
0
0
1
0
0
```

GENERICIDAD

```
//Consultas
public int cantPosiciones(){
    return t.length;
}
public int cantPosicionesOcupadas(){
    int cant = 0;
    for (int i = 0; i < cantPosiciones(); i++)
        if (t[i] != null)
            cant++;
    return cant;
}
```

0
1
0
0
0
1
0
0

26

GENERICIDAD

```
public boolean estaElemento(Object elem){
    /*Decide si la tabla mantiene un elemento con la misma
    identidad de elem, que asume no nulo*/
    boolean esta = false;
    for (int i = 0; i < cantPosiciones() && !esta; i++)
        esta = t[i] == elem;
    return esta;
}
```

0
1
0
0
0
1
0
0

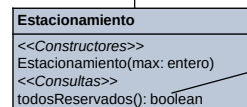
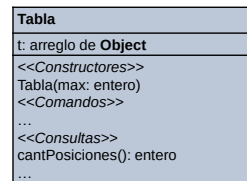
26

GENERICIDAD

```
public boolean existePosicion(int p){
    /*Decide si existe una posición p*/
    return (p >= 0 & p < cantPosiciones());
}
public Object elementoPosicion(int p){
    return t[p];
}
```

0
0
0
0
0
0
0

27



Computa true si todos los micros están llenos.

28

GENERICIDAD

```
class Estacionamiento extends Tabla{
    public Estacionamiento(int max){
        super(max);
    }
    public boolean todosReservados(){
        /*Retorna true si todos los micros estacionados tienen todos
        los asientos reservados*/
        todos = true; Micro m;
        for (int i = 0; i < cantPosiciones() && todos; i++){
            if (t[i] != null){
                todos = !t[i].hayAsientosLibres();
            }
        }
        return todos;
    }
}
```

0
0
0
0
0
0
0

29

ERROR en compilación

GENERICIDAD

```
class Estacionamiento extends Tabla{
    public Estacionamiento(int max){
        super(max);
    }
    public boolean todosReservados(){
        /*Retorna true si todos los micros estacionados tienen todos
        los asientos reservados*/
        todos = true; Micro m;
        for (int i = 0; i < cantPosiciones() && todos; i++){
            if (t[i] != null){
                m = (Micro) t[i];
                todos = !m.hayAsientosLibres();
            }
        }
        return todos;
    }
}
```

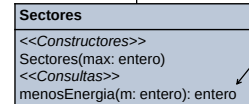
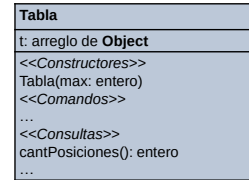
0
1
0
0
0
0
0

30

GENERICIDAD

```
class Parking{
    private Estacionamiento s;
    ...
    public administrar(){
        s = new Estacionamiento(50);
        ...
        int pos;
        ...
        pos = 0;
        Micro r = new Micro (...);
        s.insertar(r, pos);
        ...
    }
}
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99



Computa la cantidad de sectores ocupados por robots con menos de m unidades de energía.

32

GENERICIDAD

```
class FabricaJuguetes{
    private Tabla s;
    ...
    public producir(){
        s = new Tabla(50);
        ...
        int pos;
        ...
        pos = 0;
        Robot r = new Robot(...);
        s.insertar(r, pos);
        ...
    }
}
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

GENERICIDAD

```
class FabricaJuguetes{
    private Sectores s;
    ...
    public producir(){
        s = new Sectores(50);
        ...
        int pos;
        ...
        pos = 0;
        Robot r = new Robot(...);
        s.insertar(r, pos);
        int cont = s.menosEnergia(100);
        ...
    }
}
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

34